

Week 7 Code Optimization Evidence

What we mainly did this week

This week, our main work was software code optimization. We did not only design pages. We checked how the interface should connect with the backend model code, and we improved the U-Net training script so it is easier to test and use later.

The front-end screenshots show what the user sees. The code snippets show what happens behind the interface. At this stage, the UI and backend are not fully connected yet, but the core model training module is clearer than before.

UI side	Backend side	This week result
Model cards, material database, image upload, parameter panel, and result area.	Dataset reading, image resizing, U-Net model training, IoU evaluation, and best model saving.	We clarified the connection and cleaned the training code for later server integration.

1. Model version page and saved model checkpoint



Related code part

```
if mean_iou > best_iou:
    best_iou = mean_iou
    torch.save(model.state_dict(), model_save_path)
    print(f" Best model saved, IoU = {best_iou:.4f}")

return {'best_iou': best_iou, 'model_path': model_save_path}
```

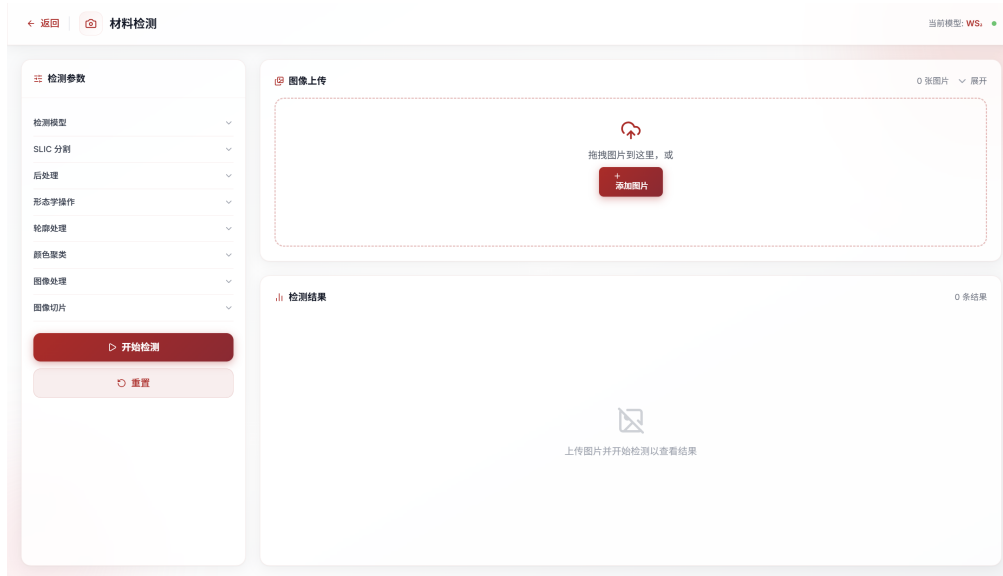
Explanation

The model version page shows different material models, such as WS₂, MoS₂, and graphene. In the future, each card can point to a saved model file.

This code saves the best U-Net model according to validation IoU. That is why it is related to the model cards. A model card can show the model version and performance, while the backend stores the actual checkpoint file.

Our work this week was to make this saving process clear, so the model can be managed more easily after training.

2. Detection workspace and image loading logic



Related code part

```
image = cv2.imread(img_path)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)

target_size = (512, 512)
image = cv2.resize(image, target_size)
mask = cv2.resize(mask, target_size, interpolation=cv2.INTER_NEAREST)
mask[mask > 0] = 1
```

Explanation

The detection workspace allows the user to upload microscope images. The current code uses a dataset loader for training, but the same idea is needed when a user uploads one image for detection.

The code reads the image, changes the color order, resizes it to 512 x 512, and changes the mask into a binary label. This helps the model receive stable input.

This week, we checked this input process because unstable image size or mask format can easily cause errors during model training and later prediction.

3. Parameter panel and preprocessing boundary

检测参数

材料类型
WS

SLIC 分割
SLIC 分块数: 3000
SLIC 紧凑度: 5.0

后处理
最小面积: 600
前 N 个输出: 5

形态学操作
形态学核大小: 3
形态学迭代次数: 2

轮廓处理
轮廓近似 Epsilon: 0.0001

颜色聚类
MeanShift 参数带宽: 7

图像处理

Related code part

```
def train_unet(train_images_dir,
               train_masks_dir,
               val_images_dir,
               val_masks_dir,
               epochs=50,
               batch_size=4,
               lr=0.001,
               model_save_path='best_model.pth'):

    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                              std=[0.229, 0.224, 0.225])
    ])
```

Explanation

The parameter panel shows many detection settings, including SLIC, post-processing, morphology, contour processing, color clustering, and image processing.

The provided Python file mainly includes U-Net training and basic preprocessing. It does not yet include the full SLIC, Mean Shift, morphology, or contour code. This is an important point we clarified this week.

So the current relationship is: the UI already prepares these settings, while the backend code currently supports the core U-Net model part. The next step is to connect more post-processing functions to the interface.

4. Material detection model and U-Net structure



Related code part

```
model = smp.Unet(
    encoder_name="resnet34",
    encoder_weights="imagenet",
    in_channels=3,
    classes=2,
)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
```

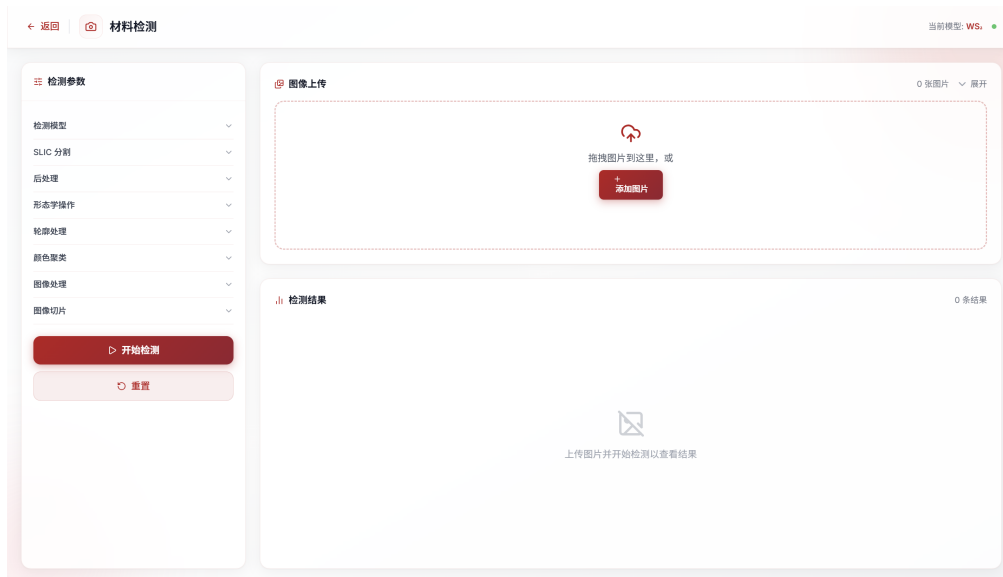
Explanation

The material database helps users understand different 2D materials before choosing a model. The backend model is the part that actually learns the material region from microscope images.

This code creates a U-Net model with a ResNet34 encoder. The output has two classes: background and material area. This matches our current segmentation goal.

This week, we kept the model structure stable and focused on making the training code easier to reuse, instead of randomly changing to a new model.

5. Result accuracy and IoU evaluation



Related code part

```
intersection = np.logical_and(pred, mask).sum()
union = np.logical_or(pred, mask).sum()

if union > 0:
    iou = intersection / union
    ious.append(iou)

mean_iou = np.mean(ious) if ious else 0.0
print(f"Epoch {epoch + 1}: Loss {avg_loss:.4f}, Val IoU {mean_iou:.4f}")
```

Explanation

The detection page has a result area. For real model testing, we need a metric to decide whether the segmentation result is good or bad.

This code calculates IoU between the predicted mask and the real mask. IoU is simple to understand: it checks how much the predicted material region overlaps with the correct material region.

This week, we used IoU as the main performance check, so our optimization is not only based on visual feeling but also based on a numeric result.

6. What is still not finished

The current Python code is mainly a U-Net training script. It is not the full website backend yet. The login system, image upload API, SLIC and Mean Shift processing, UI parameter transfer, and result visualization still need more integration work.

However, this week we made the core model code more organized. This is useful because the website can only work well after the backend model part is stable enough.

Appendix A - Complete code with English comments

This is the complete code logic used in this evidence file. The comments and printed messages are written in English for the report.

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms
import segmentation_models_pytorch as smp
import cv2
import numpy as np
import os

# ===== Dataset class =====
class CustomDataset(Dataset):
    def __init__(self, images_dir, masks_dir, transform=None):
        self.images_dir = images_dir
        self.masks_dir = masks_dir
        self.transform = transform
        # Support both .jpg and .png image files.
        self.image_names = [f for f in os.listdir(images_dir) if f.endswith((''.jpg', '.png'))]

    def __len__(self):
        return len(self.image_names)

    def __getitem__(self, idx):
        img_name = self.image_names[idx]
        img_path = os.path.join(self.images_dir, img_name)

        # Match the mask file name. In our dataset, masks are saved as .png files.
        if img_name.endswith(''.jpg'):
            mask_name = img_name.replace(''.jpg', '.png')
        else:
            mask_name = img_name
        mask_path = os.path.join(self.masks_dir, mask_name)

        image = cv2.imread(img_path)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)

        target_size = (512, 512)
        image = cv2.resize(image, target_size)
        mask = cv2.resize(mask, target_size, interpolation=cv2.INTER_NEAREST)

        # Convert the mask into two classes: 0 for background and 1 for material area.
        mask[mask > 0] = 1

        if self.transform:
            image = self.transform(image)

        mask = torch.from_numpy(mask).long()
        return image, mask

# ===== Evaluation function =====
def evaluate_model(model, dataloader, device):
    model.eval()
    all_iious = []
    with torch.no_grad():
        for images, masks in dataloader:
            images, masks = images.to(device), masks.to(device)
            outputs = model(images)
            preds = torch.argmax(outputs, dim=1)

            for i in range(preds.shape[0]):
                pred = preds[i].cpu().numpy()
                mask = masks[i].cpu().numpy()

                intersection = np.logical_and(pred, mask).sum()
                union = np.logical_or(pred, mask).sum()

                if union > 0:
                    iou = intersection / union
                    all_iious.append(iou)

    if not all_iious:
        return 0.0
    return np.mean(all_iious)

# ===== Training function called by server.py =====
def train_unet(train_images_dir,
               train_masks_dir,
               val_images_dir,
               val_masks_dir,
               epochs=50,
               batch_size=4,
               lr=0.001,
               model_save_path='best_model.pth'):
    """
    Train a U-Net segmentation model.

    Parameters:
        train_images_dir: folder path for training microscope images
        train_masks_dir: folder path for training masks
    """
```



```

    val_images_dir:    folder path for validation microscope images
    val_masks_dir:     folder path for validation masks
    epochs:            number of training epochs
    batch_size:        batch size
    lr:                learning rate
    model_save_path:   file path for saving the best model

Returns:
    dict: {'best_iou': float, 'model_path': str}
"""
# Image preprocessing. The normalization values are suitable for the ResNet34 encoder.
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

# Create datasets and data loaders.
train_dataset = CustomDataset(train_images_dir, train_masks_dir, transform=transform)
val_dataset = CustomDataset(val_images_dir, val_masks_dir, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

# Create the U-Net model. ResNet34 is used as the encoder.
model = smp.Unet(
    encoder_name="resnet34",
    encoder_weights="imagenet",
    in_channels=3,
    classes=2,
)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=lr)

best_iou = 0.0
for epoch in range(epochs):
    # Training stage.
    model.train()
    total_loss = 0
    for images, masks in train_loader:
        images, masks = images.to(device), masks.to(device)
        outputs = model(images)
        loss = criterion(outputs, masks)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
    avg_loss = total_loss / len(train_loader)

    # Validation stage.
    model.eval()
    ious = []
    with torch.no_grad():
        for images, masks in val_loader:
            images, masks = images.to(device), masks.to(device)
            outputs = model(images)
            preds = torch.argmax(outputs, dim=1)

            for i in range(preds.shape[0]):
                pred = preds[i].cpu().numpy()
                mask = masks[i].cpu().numpy()
                inter = np.logical_and(pred, mask).sum()
                union = np.logical_or(pred, mask).sum()

                if union > 0:
                    ious.append(inter / union)

    mean_iou = np.mean(ious) if ious else 0.0
    print(f"Epoch {epoch + 1}: Loss {avg_loss:.4f}, Val IoU {mean_iou:.4f}")

    # Save the best model based on validation IoU.
    if mean_iou > best_iou:
        best_iou = mean_iou
        torch.save(model.state_dict(), model_save_path)
        print(f" Best model saved, IoU = {best_iou:.4f}")

return {'best_iou': best_iou, 'model_path': model_save_path}

# Allow this script to run directly for training with default folders.
if __name__ == '__main__':
    train_images_dir = 'total_dataset/train/images'
    train_masks_dir = 'total_dataset/train/masks'
    val_images_dir = 'total_dataset/val/images'
    val_masks_dir = 'total_dataset/val/masks'

    if not os.path.exists(train_images_dir):
        print(
            f"Error: Please create the dataset folder '{train_images_dir}' "
            "and put training images into it first."
        )
        exit(1)

    result = train_unet(

```

```
train_images_dir=train_images_dir,  
train_masks_dir=train_masks_dir,  
val_images_dir=val_images_dir,  
val_masks_dir=val_masks_dir,  
epochs=50,  
batch_size=4,  
lr=0.001,  
model_save_path='best_model.pth'  
)  
print(f"Training completed. Best IoU: {result['best_iou']:.4f}")
```

Appendix B - Detailed explanation of the complete code

Imports

The code imports PyTorch for deep learning, OpenCV for image reading and resizing, NumPy for numeric calculation, and segmentation_models_pytorch for building the U-Net model. This makes the training script shorter than writing a U-Net from the beginning.

CustomDataset

This class tells the DataLoader how to find images and masks. It supports both jpg and png image files, which is useful because our microscope images may not always have the same file type.

Image and mask matching

For each image, the code tries to find the matching mask. If the image is jpg, the mask name is changed to png. This is reasonable because masks are better saved as png, since png keeps labels more clearly.

Image preprocessing

The image is read by OpenCV, converted from BGR to RGB, and resized to 512 x 512. The mask is also resized, but it uses nearest-neighbor interpolation so the labels do not become mixed values.

Binary mask setting

The line `mask[mask > 0] = 1` changes the mask into two classes. Class 0 means background, and class 1 means 2D material region. This matches the current goal of separating material from background.

Transforms

The image is converted into a tensor and normalized with ImageNet mean and standard deviation values. This fits the ResNet34 encoder because the encoder starts from pretrained ImageNet weights.

Evaluation function

The `evaluate_model` function runs the model without training and calculates IoU for each prediction. It uses `torch.no_grad()` so the evaluation is faster and does not store unnecessary gradient information.

train_unet function

This function is the main part of the code. It takes dataset paths and training settings as input. This design is better than writing fixed paths everywhere because the server can call this function later with different folders.

DataLoaders

The training DataLoader shuffles the data because random order helps training. The validation DataLoader does not shuffle because validation only checks performance.

U-Net model

The model is U-Net with a ResNet34 encoder. In our project, U-Net is used to segment 2D material regions from microscope images. The model output has two classes: background and material.

Device selection

The code uses CUDA if a GPU is available. If not, it uses CPU. This makes the code easier to run on different computers.

Loss and optimizer

CrossEntropyLoss is used because this is a classification problem for each pixel. Adam is used as the optimizer because it is common and easy to use for this kind of training task.

Training loop

For each epoch, the model predicts masks, calculates loss, clears old gradients, updates the weights, and records average loss. This lets us know whether the model is learning during training.

Validation loop

After each epoch, the model is tested on validation images. The code calculates IoU and prints it. This is important because low training loss alone does not always mean the model works well on new images.

Best model saving

The best model is saved only when validation IoU improves. This is better than saving the last model, because the last epoch may not be the best result.

Direct running block

The last part allows the script to be run directly. It checks whether the dataset folder exists and then starts training with default paths. This is useful for testing before linking the script with the website.

Connection to the UI

When this code is connected with the web system, the model selection page can choose a saved checkpoint, and the detection page can send uploaded images to the backend for segmentation.

Current limitation

This code does not include full API connection, login logic, SLIC, Mean Shift, morphology, contour filtering, or front-end result drawing. These are the next integration tasks after the U-Net part becomes stable.